Schéma de Base de Données

Snake AI — Documentation technique

Vue d'ensemble

La base de données SQLite (**snake.db**) est créée automatiquement au démarrage du backend via SQLAlchemy ORM. Elle contient **5 tables**. Le fichier **snake.db** est généré à la racine du projet (pas dans *backend/*).

Schéma relationnel

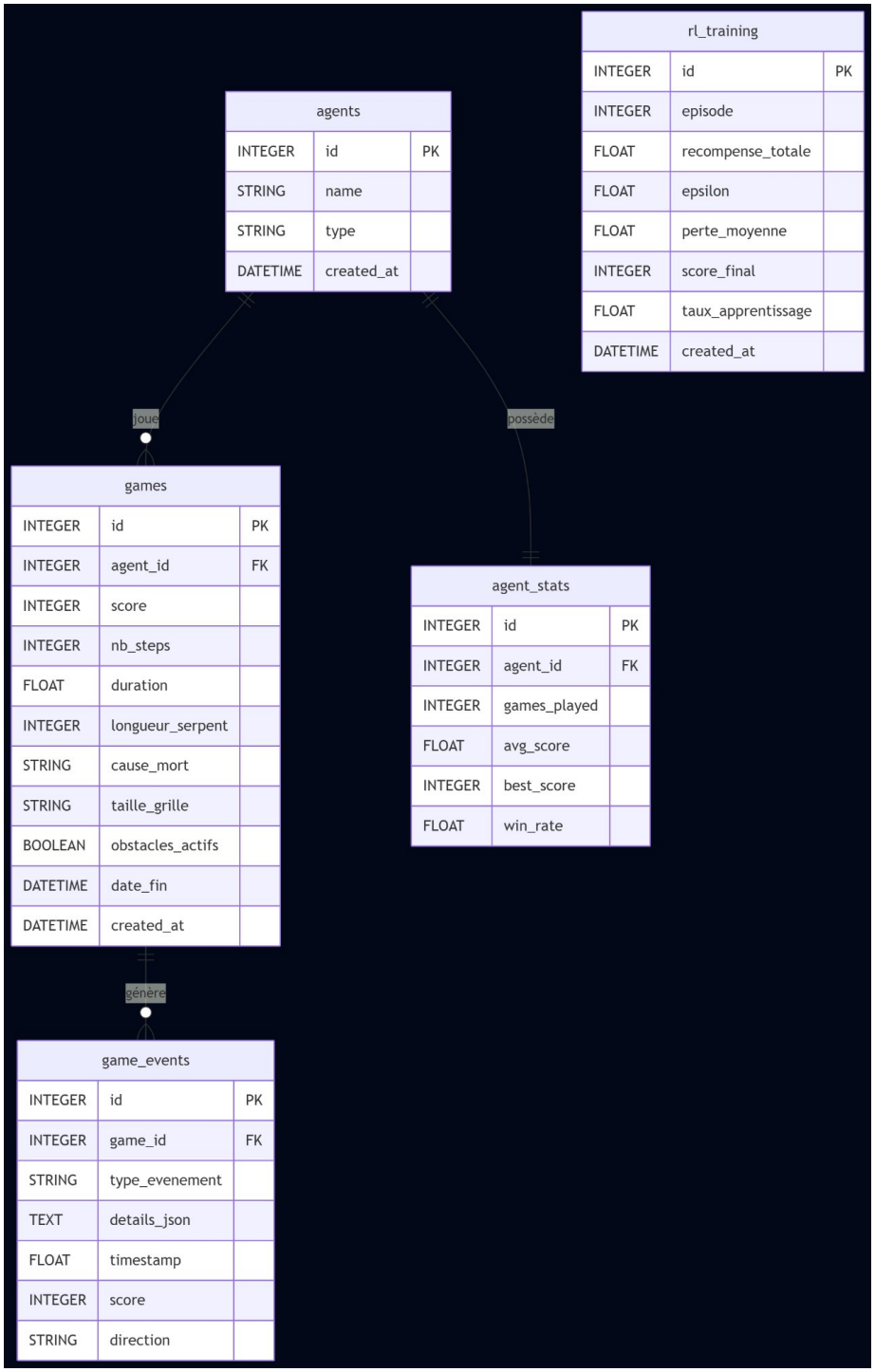


Diagramme entité-relation généré via Mermaid Live Editor

Description des tables

Table agents

Référentiel des agents du système. **3 agents sont créés automatiquement** au premier lancement.

Colonne	Type SQLAlchemy	Description
id	Integer, PK, autoincrement	Identifiant unique
name	String	Nom de l'agent
type	String	Type : human, astar, rl
created_at	DateTime	Date de création

Données initiales (seed) :

id	name	type
1	Agent Humain	human
2	Agent A*	astar
3	Agent Q-Learning	rl

Table games

Enregistre chaque partie jouée par un agent.

Colonne	Type	Nullable	Description
id	Integer, PK	Non	Identifiant unique
agent_id	Integer, FK	Non	Référence à agents.id
score	Integer	Non	Score final
nb_steps	Integer	Non	Nombre de mouvements
duration	Float	Non	Durée en secondes
longueur_serpent	Integer	Oui	Longueur finale du serpent
cause_mort	String	Oui	mur, corps, obstacle, battle
taille_grille	String	Oui	Ex. 25x25
obstacles_actifs	Boolean	Oui	Obstacles présents
date_fin	DateTime	Oui	Horodatage de fin de partie
created_at	DateTime	Non	Horodatage d'insertion

Table game_events

Trace les événements survenus pendant une partie (granularité fine pour analyse).

Colonne	Type	Nullable	Description
id	Integer, PK	Non	Identifiant unique
game_id	Integer, FK	Non	Référence à games.id
type_evenement	String(50)	Oui	move, food, collision
details_json	Text	Oui	Données JSON libres (position, direction…)
timestamp	Float	Non	Secondes depuis le début de la partie
score	Integer	Oui	Score au moment de l'événement
direction	String(10)	Oui	HAUT, BAS, GAUCHE, DROITE

Table agent_stats

Statistiques agrégées par agent, mises à jour après chaque partie. Contrainte : **UNIQUE sur agent_id** (1 ligne par agent).

Colonne	Type SQLAlchemy	Description
id	Integer, PK	Identifiant unique
agent_id	Integer, FK UNIQUE	Référence à agents.id
games_played	Integer	Nombre total de parties
avg_score	Float	Score moyen
best_score	Integer	Meilleur score
win_rate	Float	Taux de survie (score > 0)

Table rl_training

Historique des épisodes d'entraînement Q-Learning. **Table indépendante** (ne référence pas agents).

Colonne	Type	Nullable	Description
id	Integer, PK	Non	Identifiant unique
episode	Integer	Non	Numéro de l'épisode (1 à N)
recompense_totale	Float	Non	Récompense cumulée sur l'épisode
epsilon	Float	Non	Taux d'exploration
perte_moyenne	Float	Non	Erreur TD moyenne (différence Bellman)
score_final	Integer	Non	Score final de l'épisode
taux_apprentissage	Float	Non	Alpha (learning rate, typiquement 0.1)
created_at	DateTime	Oui	Date d'enregistrement

Relations

Relation	Type	Description
agents → games	1 à N	Un agent joue plusieurs parties
games → game_events	1 à N	Une partie génère plusieurs événements
agents → agent_stats	1 à 1	Contrainte UNIQUE sur agent_id
rl_training	Indépendante	Ne référence pas agents

Requêtes typiques

Statistiques comparatives A* vs RL

```
SELECT a.type, ast.avg_score, ast.best_score, ast.games_played, ast.win_rate
FROM agents a
JOIN agent_stats ast ON a.id = ast.agent_id
WHERE a.type IN ('astar', 'rl');
```

Historique des 20 dernières parties A*

```
SELECT g.score, g.nb_steps, g.cause_mort, g.created_at
FROM games g
JOIN agents a ON g.agent_id = a.id
WHERE a.type = 'astar'
ORDER BY g.created_at DESC
LIMIT 20;
```

Évolution epsilon sur les épisodes RL

```
SELECT episode, epsilon, score_final
FROM rl_training
ORDER BY episode ASC;
```

Fichiers ORM SQLAlchemy

Fichier	Table mappée
backend/models/agent.py	agents
backend/models/game.py	games
backend/models/game_event.py	game_events
backend/models/stats.py	agent_stats

Fichier	Table mappée
backend/models/rl_training.py	rl_training
backend/database.py	Engine, SessionLocal, seed des 3 agents